Tarea 2: Adquisición con BME688 y procesamiento local

Interfaz gráfica PyQt, comunicación serial y cómputo en ESP32

CC5328-1: Sistemas Embebidos y Sensores

Universidad de Chile

Semestre 2026-2

Profesor	Luciano Radrigan
Auxiliar	Lucas Llort
Modalidad	Trabajo en equipo; revisión de código y video de funcionamiento
Fecha de entrega	**Viernes 6 de noviembre de 2026, 23:59 h**
Formato de entrega	Repositorio Git o archivo ZIP, README y video (archivo o enlace)
Escala de notas	1,0 a 7,0; nota base 1,0 + hasta 6,0 puntos

1 Introducción y objetivos

La Tarea 1 abordó la generación de datos simulados en el ESP32 y su visualización mediante una interfaz gráfica en el computador. En esta segunda tarea se incorporará un **sensor físico BME688**, del que se obtendrán mediciones reales de **temperatura, humedad relativa y presión atmosférica**.

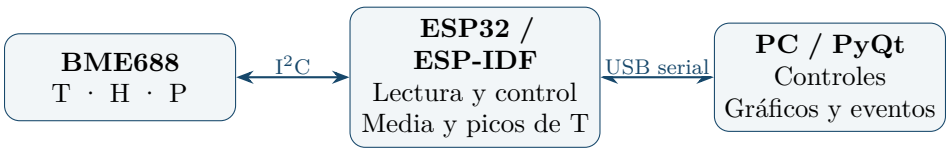
El sistema deberá permitir que una aplicación **Python/PyQt** controle de forma remota la adquisición y transmisión de datos. Adicionalmente, el ESP32 ejecutará operaciones sobre la señal de temperatura **antes de transmitir sus resultados**, para diferenciar el procesamiento embebido de los cálculos realizados en el computador.

Objetivos de aprendizaje:

- Integrar un sensor físico mediante I^2C y obtener medidas válidas.
- Diseñar comandos bidireccionales para reconfigurar un dispositivo en tiempo de ejecución.
- Seleccionar dinámicamente qué variables se transmiten, sin confundir adquisición con transmisión.
- Implementar una media temporal y detección de picos de temperatura en el ESP32.
- Visualizar datos, parámetros y eventos en una GUI no bloqueante y validar el sistema experimentalmente.

2 Descripción del sistema

El sistema se divide en tres bloques: sensor BME688, firmware ESP32 y aplicación de escritorio. La comunicación BME688–ESP32 será mediante I^2C; la comunicación ESP32–PC será serial por USB (UART/USB-Serial, según la tarjeta).



Alcance: aunque el BME688 también incorpora medición de gas, **esta tarea utiliza únicamente temperatura (T), humedad relativa (H) y presión (P)**. No se solicita implementar el calentador de gas ni algoritmos de calidad del aire. Las señales deben provenir de lecturas reales del BME688; no se permite reemplazarlas por valores aleatorios en la operación normal.

2.1 Módulo ESP32: adquisición de datos reales

1. Configurar el BME688 y demostrar lecturas válidas de T en °C, H en %HR y P en hPa.
2. Implementar manejo de fallos de inicialización y de lectura; no enviar valores inventados como si fueran mediciones válidas.
3. Permitir una **frecuencia de muestreo común** para las variables del sensor, configurable desde la GUI con al menos estos valores: **0,5; 1; 2 y 5 Hz** (valor inicial sugerido: 1 Hz).
4. Aplicar los cambios sin reiniciar el ESP32. Las tasas exigidas corresponden al modo de medición de T/H/P sin lectura de gas; el firmware debe respetar el tiempo necesario para completar cada conversión.

2.2 Selección de variables transmitidas

La GUI deberá permitir seleccionar libremente una o más variables mediante tres controles independientes: **Temperatura**, **Humedad** y **Presión**. Son obligatorias las **siete combinaciones no vacías**:

T	H	P	Combinación transmitida
✓	–	–	Solo temperatura
–	✓	–	Solo humedad
–	–	✓	Solo presión
✓	✓	–	Temperatura y humedad
✓	–	✓	Temperatura y presión
–	✓	✓	Humedad y presión
✓	✓	✓	Las tres variables

El ESP32 deberá transmitir **solo las mediciones seleccionadas** en cada paquete de adquisición. Una selección vacía debe rechazarse o impedirse en la GUI. **Importante:** excluir temperatura de la transmisión *no* impide su adquisición interna cuando está habilitado el cálculo de promedio o detección de picos.

3 Procesamiento local en el ESP32

Las siguientes funciones deben estar implementadas en el firmware y configurarse desde PyQt. **Ambas operan exclusivamente sobre la temperatura real del BME688** y pueden activarse o desactivarse de manera independiente.

3.1 Promedio temporal de temperatura

El ESP32 calculará una **media móvil temporal** utilizando todas las mediciones de temperatura disponibles en los últimos W segundos:

$$\overline{T}_W(t) = \frac{1}{N_W(t)} \sum_{i:\, t-W<t_i\le t} T_i, \qquad N_W(t) \ge 1.$$

- La ventana W se configura desde la GUI. Valores mínimos requeridos: **5, 10, 30 y 60 segundos**; valor inicial sugerido: 10 s.

- La GUI debe ofrecer controles para **habilitar/deshabilitar** el promedio y modificar W durante la adquisición.
- El resultado debe calcularse **en el ESP32** y transmitirse etiquetado como `temp_avg_c`, incluso si la temperatura cruda no se seleccionó para transmisión.
- Al cambiar W, el resultado se actualizará a la nueva ventana usando las muestras retenidas disponibles. Durante el arranque, se calcula con las muestras existentes dentro de la ventana y se informa si aún no se completó una ventana temporal entera.

3.2 Detección de picos de temperatura

El ESP32 detectará **máximos locales que superen un umbral configurable** θ (en °C). Una definición mínima aceptada para tres muestras consecutivas es:

$$T_{k-1} > T_{k-2}, \qquad T_{k-1} \geq T_k, \qquad T_{k-1} \geq \theta.$$

- Habilitar/deshabilitar la detección y ajustar θ desde PyQt, con resolución de **0,1 °C** (valor inicial sugerido: 28,0 °C; debe ser modificable).
- Ante un pico válido, enviar un **evento independiente** con el valor del pico, el umbral aplicado y su marca de tiempo. No enviar repetidamente el mismo pico.
- La GUI debe mostrar un historial de eventos de pico y el valor actual del umbral.
- Se permite aplicar filtrado o separación mínima entre eventos para reducir detecciones espurias, siempre que la estrategia se explique y pueda verificarse.

Distinción conceptual: superar el umbral no es, por sí solo, detectar un máximo local. El criterio utilizado por el equipo debe describirse claramente y probarse con una secuencia de temperatura conocida.

4 Aplicación de escritorio (Python/PyQt)

La interfaz deberá integrar las siguientes funcionalidades:

- **Conexión:** enumerar/seleccionar puerto, configurar velocidad serial, conectar/desconectar y mostrar estado del dispositivo.
- **Configuración de adquisición:** seleccionar la frecuencia, las variables T/H/P que se transmitirán y enviar los cambios al ESP32.
- **Procesamiento local:** activar/desactivar promedio y picos; controlar la ventana W y el umbral θ; mostrar la configuración que el ESP32 confirmó.
- **Visualización:** mostrar valores numéricos actualizados y gráficos históricos de T, H y P, con sus unidades; permitir reconocer variables deshabilitadas para transmisión.
- **Resultados:** mostrar el promedio recibido desde el ESP32, identificar eventos de picos con valor y tiempo, e indicar cuándo una lectura es inválida o no se recibe información.
- **Robustez:** no bloquear la GUI durante la lectura serial; gestionar errores de puerto, desconexiones y comandos no aceptados.

La GUI **no debe reemplazar el cómputo embebido**: el promedio y la detección de picos exigidos se generan en el ESP32. Podrá recalcularse un resultado en el PC *solo para validación* y deberá identificarse como tal.

5 Protocolo serial y comportamiento esperado

Se utilizará un protocolo bidireccional documentado. Puede ser texto delimitado o JSON por línea; debe permitir diferenciar **datos**, **comandos**, **confirmaciones**, **errores** y **eventos**. Cada paquete de datos debe incluir una marca de tiempo del ESP32 y/o un número de secuencia.

Ejemplo orientativo (JSON Lines; no obligatorio):

```
GUI -> ESP32
{"type":"config","sampling_hz":2,"variables":["T","P"],
 "average":{"enabled":true,"window_s":10},
 "peak":{"enabled":true,"threshold_c":28.0}}

ESP32 -> GUI
{"type":"ack","status":"ok","sampling_hz":2,
 "variables":["T","P"]}
{"type":"sample","seq":145,"t_ms":73210,
 "temperature_c":26.4,"pressure_hpa":1012.7,
 "temp_avg_c":26.1,"avg_window_complete":true}
{"type":"peak","t_ms":75410,
 "peak_c":28.7,"threshold_c":28.0}
```

En el ejemplo, no se transmite humedad porque no está seleccionada; si se habilita un cálculo, sus resultados pueden viajar aunque **T no figure entre las variables crudas seleccionadas**. Los comandos inválidos deben recibir una respuesta de error o rechazo identificable. Se debe evitar que una modificación parcial deje configuraciones inconsistentes.

6 Pruebas obligatorias y video de demostración

Cada equipo debe preparar pruebas **reproducibles** y registrar su funcionamiento en un **video de demostración obligatorio**, que se adjuntará con la entrega. Los resultados y el procedimiento de las pruebas se resumirán en el README.

1. **Lecturas reales:** mostrar T, H y P simultáneamente e indicar unidades, cableado y frecuencia empleada.
2. **Siete combinaciones:** mostrar en el video al menos tres, incluyendo una sola variable, dos variables y las tres. Documentar pruebas de las siete combinaciones.
3. **Cambio dinámico:** alternar, por ejemplo, 1 Hz a 5 Hz sin reiniciar y verificar la periodicidad mediante marcas de tiempo o secuencias.
4. **Promedio:** cambiar W (por ejemplo, 5 s a 30 s); comparar un conjunto de resultados con un cálculo de referencia y mostrar que el ESP32 lo emite.
5. **Picos:** ajustar θ, provocar o reproducir de forma controlada un máximo local y comprobar la emisión de un evento único. La prueba unitaria puede usar una serie conocida, pero la operación normal debe permanecer conectada al sensor real.
6. **Procesamiento con T oculta:** seleccionar solo H y/o P y verificar que el promedio/eventos de temperatura siguen siendo generados en el ESP32 cuando están habilitados.
7. **Manejo de errores:** enviar una configuración inválida y demostrar el tratamiento de una desconexión/reconexión serial.

Requisitos del video: duración sugerida de **8 a 12 minutos**, en formato `MP4` o mediante un enlace accesible para revisión. Debe verse el **montaje real BME688–ESP32**, la **interfaz PyQt en ejecución** y la interacción con los controles mientras se verifican las pruebas anteriores. Se requiere una explicación breve de lo que se está mostrando (voz o rótulos claros), y el texto de la GUI y sus gráficos debe ser legible. Se permiten cortes para organizar las pruebas, siempre que no sustituyan la evidencia del funcionamiento real. Un video compuesto solo por diapositivas o capturas estáticas no reemplaza la demostración.
Si se utiliza un servicio externo para alojarlo, incluir el **enlace directo en el README** y verificar que el equipo docente pueda visualizarlo sin solicitar permisos. Si se entrega como archivo, incluirlo como `video_demostracion.mp4` en el ZIP o repositorio, de acuerdo con las restricciones de tamaño de la plataforma. La evaluación práctica se realizará **a partir del video**, sin demostración presencial obligatoria.

Seguridad y realismo experimental: la temperatura ambiental suele variar lentamente. Se recomienda utilizar un umbral cercano a la temperatura real del entorno o una fuente de calor suave y controlada, sin exponer la placa a temperaturas fuera de las especificaciones del fabricante. No es obligatorio producir un gran cambio térmico.

7 Pauta de evaluación y puntaje

La nota se calculará como **1,0 + puntos obtenidos**, con un máximo de 7,0. La rúbrica contiene 6,0 puntos: **2,4 puntos asociados a revisión de código (40 %)** y **3,6 puntos asociados al video de funcionamiento (60 %)**. No existe una segunda ponderación independiente.

Criterio	Código	Video	Total
Lecturas reales BME688 (T, H, P) y adquisición correcta	0,35	0,65	1,00
Frecuencia de muestreo configurable y verificable	0,30	0,40	0,70
Selección de variables y protocolo bidireccional	0,35	0,45	0,80
Promedio temporal de temperatura en ESP32	0,40	0,40	0,80
Detección de picos y eventos en ESP32	0,35	0,45	0,80
GUI: controles, gráficos y estados	0,15	0,65	0,80
Robustez, comandos inválidos y reconexión	0,25	0,45	0,70
Modularidad, README y evidencia de pruebas	0,25	0,15	0,40
Subtotal evaluable	**2,40**	**3,60**	**6,00**
Nota base			1,00
NOTA MÁXIMA			**7,00**

8 Instrucciones de entrega

Se entregará un enlace al repositorio Git o un ZIP. Debe incluir, como mínimo:

- `esp32_firmware/`: proyecto ESP-IDF, fuentes, `CMakeLists.txt` y configuración necesaria.
- `gui_python/`: aplicación PyQt ejecutable y archivos asociados.
- `requirements.txt`: dependencias Python (PyQt5/6, pyserial y biblioteca de gráficos).
- `README.md`: instalación para ESP32 y PC, conexión física y pines, ejecución, formato del protocolo, decisiones de diseño, capturas, resultados de pruebas y acceso al video si se entrega mediante enlace.
- **Video de demostración obligatorio**: archivo `video_demostracion.mp4` o enlace directo de visualización en el README. Debe estar disponible hasta finalizar la evaluación.
- Opcional: carpeta `tests/` con pruebas de media temporal, picos y validación de mensajes.

Requisitos técnicos: firmware en C/C++ con ESP-IDF; GUI en Python 3 con PyQt5 o PyQt6; comunicación serial mediante `pyserial`; biblioteca gráfica libre (`pyqtgraph` o `matplotlib`). Puede usarse la *BME68x Sensor API* de Bosch para manejar el sensor. La elección de pines I^2C se debe documentar y debe ser compatible con la placa utilizada.

9 Referencias y recomendaciones

- Bosch Sensortec, *BME688 Datasheet* (documento PDF oficial).
- Bosch Sensortec, *BME68x SensorAPI* (repositorio oficial).
- Espressif, *ESP-IDF Programming Guide* (seleccionar el chip utilizado).

Se recomienda probar por etapas: **I^2C y lecturas válidas → serial → GUI → reconfiguración → promedio y picos → robustez**. Documentar el formato exacto de cada paquete y no asumir que un comando se aplicó hasta recibir su confirmación.